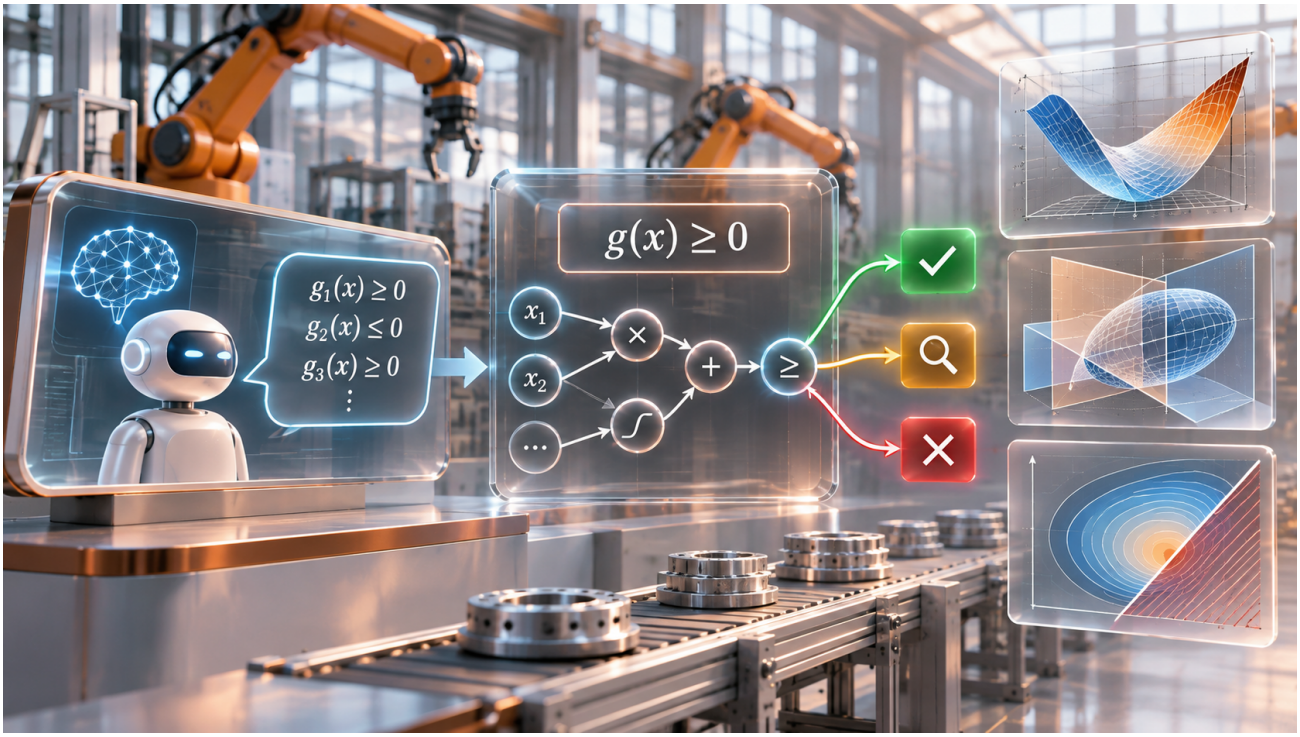
Automatic Verification of Agent-Generated Constraints

Reusable nonlinear guard evaluation for controlled industrial decision workflows.



Cover artwork: conceptual manufacturing and computational verification illustration, not observed plant data.

Recorded experiment	Result
Versioned mathematical guard handles	1,001 (1,000 original + one immutable revision)
Adversarial guard-input decisions	100,000
Mathematical evaluation	Scalar + packed batch via original paceval SDK
Decision outcomes	ACCEPT / REJECT / REVIEW

The experiments use synthetic normalized manufacturing setpoints. No general formal correctness, deployed machine safety or live autonomous-agent behavior is claimed.

1. Industrial use case and current state of the art

An industrial engineering assistant might propose a robot-workspace target, a machine setpoint or a thermal operating envelope. In an automated factory, these proposed numerical values require independent gating before any control action. A textual or model-generated claim of validity is not itself authorization, and the numerical formula is only one part of the engineering safety boundary.

The illustrative use case is a hypothetical manufacturing cell. An upstream planner selects a constraint from a controlled, typed registry. A C++ application first rejects unauthorized actions, invalid units, nonfinite values and numbers outside the declared input range. The mathematical kernel then computes a signed guard margin from normalized setpoints. The host routes results to an explicit acceptance, rejection or human-review state.

Present-day technical approaches

Formal SMT/SAT-based tools can reason about logical conditions within particular theories, while numerical expression engines evaluate defined mathematical functions at selected arguments. These are distinct capabilities. SMT-LIB 2.7 formalizes satisfiability-modulo-theories reasoning, and the Z3 guidance documents limitations in nonlinear arithmetic. NIST AI risk guidance motivates risk-aware evaluation of AI-assisted workflows; MCP tool specifications provide a machine-readable contract and request interface, not a guarantee that an invoked mathematical decision is safe.

References: SMT-LIB 2.7 (2025); Microsoft Z3 arithmetic guide; NIST AI 600-1 (2024); MCP tools specification (2026-07-28). Full links are provided on page 10.

2. Experimental question and system responsibilities

Can a fixed allowlisted catalogue of 1,000 nontrivial mathematical inequalities be represented as persistent paceval computation graphs, repeatedly evaluated for boundary-focused proposed machine inputs, and safely connected to a three-way host routing mechanism that keeps authorization separate from arithmetic?

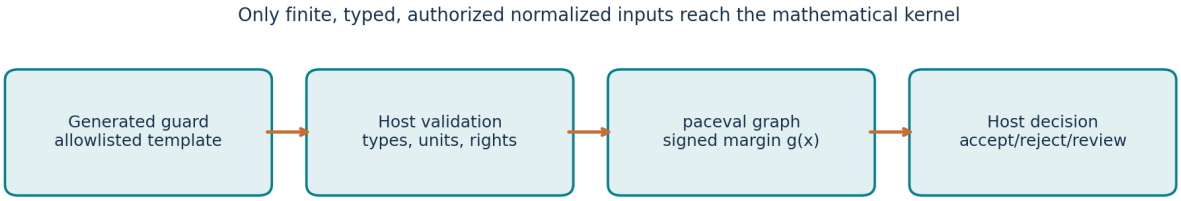


Figure 1. The actual implemented boundary from allowlisted guard definition to checked numeric execution and host-side routing.

Interface ownership

paceval mathematical engine	C++ host / surrounding application
Create 1,001 persistent graph handles	Template registry, version control and action permissions
Evaluate scalar signed inequalities	Input typing, finite/range/unit validation
Execute packed independent input batches	Apply boundary-routing policy and manage audit records
Return numerical margins for decisions	C++ reference, scenario design and CSV export

The engine does not parse free-form LLM instructions, authorize a machine action or prove that a guard is valid for every possible input. The experiment also does not operate a live MCP server or physical robot.

3. Defined mathematical constraint models

All quantities x, y and z are dimensionless normalized industrial setpoints admitted only on the defined finite range [-2,2]. Each admissible mathematical guard is represented by a signed margin g(x,y,z), with nonnegative values satisfying the underlying inequality.

Five implemented guard families

g1 = c - x [upper threshold]

g2 = c - (x/a)^2 - (y/b)^2 [elliptical workspace]

g3 = c - x^2 - k*y^2 - 0.1*z^2 [quadratic budget]

g4 = c + a*x - b*y + k*z [affine envelope]

g5 = c - exp(k*x) + a*cos(y) - b*z^2 [nonlinear envelope]

Two hundred formula configurations are generated for each family, with explicitly frozen values of a, b, c and k in the released guard registry. A controlled second version of the first guard changes its threshold and produces an additional computation handle; the earlier handle remains accessible.

Numerical decision policy

ACCEPT if g > +1e-8; REJECT if g < -1e-8; REVIEW otherwise.

This three-way routing is deliberate. A point extremely close to a guard boundary is not silently approved. The review band is a declared policy choice, not a mathematically certified error enclosure. All host-level schema checks precede the calculation.

4. Synthetic industrial evaluation workload

The released registry contains exactly 1,000 original guard expressions and one independently created revision. A fixed xorshift-based generator produces 100 test arguments for each original guard, yielding 100,000 logged decisions. Half of the input positions are deliberately placed close to an analytically derived zero-margin surface; the remaining inputs explore a wider admitted region.

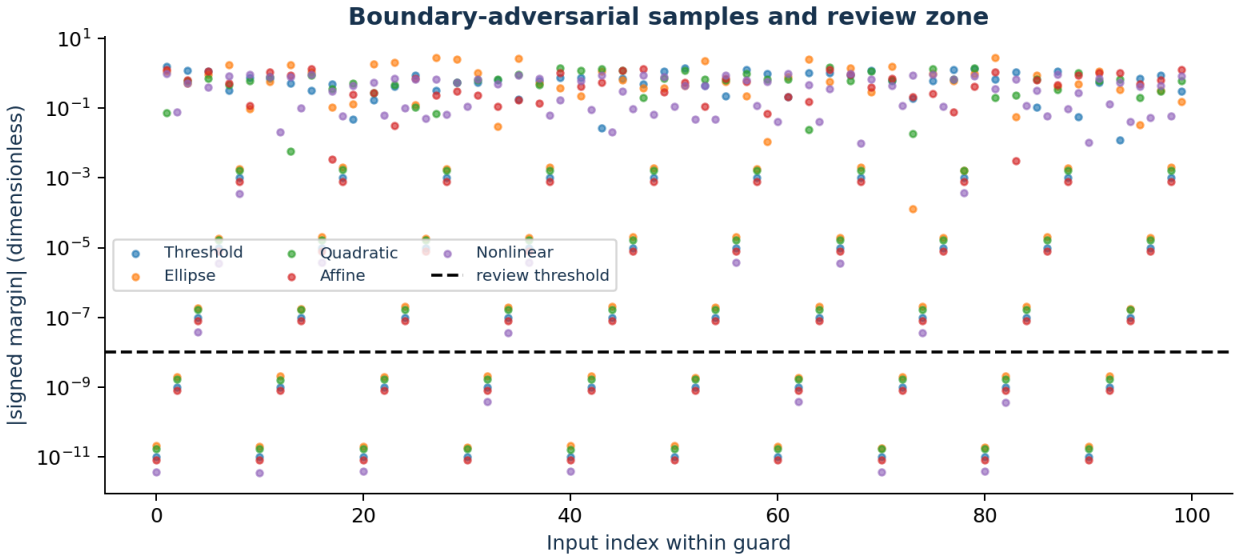


Figure 3. Selected guard families are evaluated with explicit emphasis on the numerical switching region around the inequality boundary.

The host separately tests four invalid-contract cases: mismatched units, unknown action, nonfinite numerical input, and out-of-range values. These are rejected by the host and are not presented as paceval mathematical proofs.

A finite deterministic sample is appropriate for executable software evaluation, but not a proof over a continuous parameter domain.

5. Executed paceval computation and audit status

Recorded experimental quantity	Actual result
Original guard expressions	1,000
Persistently registered mathematical graphs	1,001
Adversarial pointwise constraint inputs	100,000
Scalar mathematical evaluations, including tests and surface	103,723
Packed batch checks	8,000
Independent 80-digit sample checks	632; completed successfully
Host schema/authorization rejection tests	4
CMake / CTest integration	1 / 1 passed

The application uses the real paceval initialization, graph creation, scalar computation, packed batch evaluation, error checking and handle release routines. Exact mathematical expressions and test parameters are recorded in the project CSVs. An independent native C++ formula evaluator and a separately implemented 80-digit arithmetic check support the technical audit; details of numerical agreement reside in the machine-readable verification files, not in the industrial outcome discussion.

This is evidence that a controlled mathematical kernel can compute the defined guard functions for the evaluated inputs. It is not a claim of formal satisfiability, universal numerical guarantees, equipment certification, or autonomous industrial authorization.

6. Industrial decisions and routing behavior

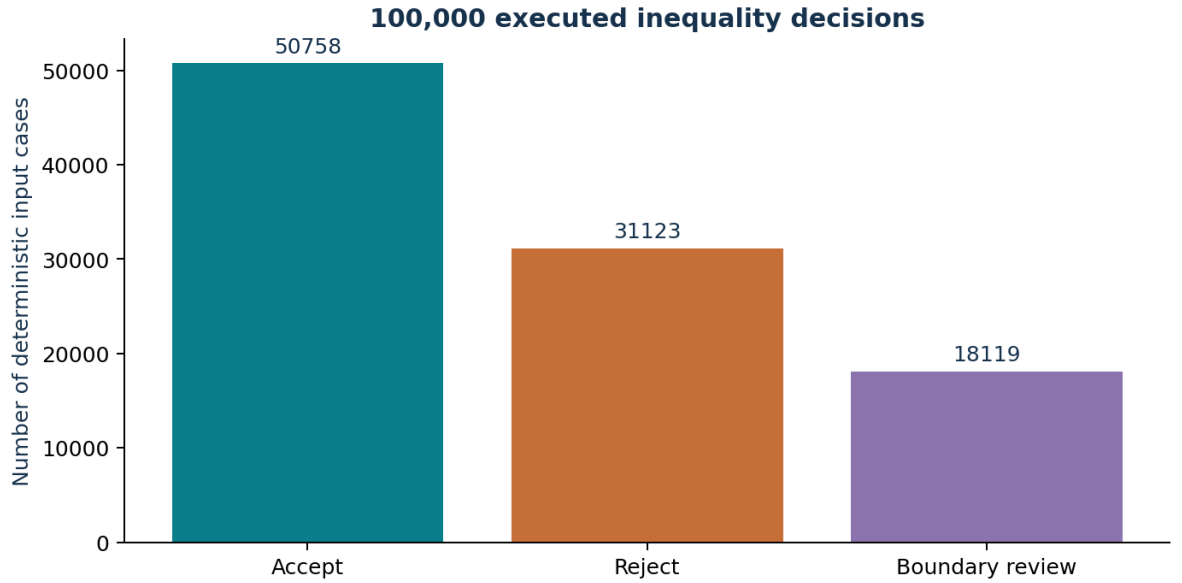


Figure 2. Actual application decisions across 100,000 synthetic guard-input pairs: 50,758 accept, 31,123 reject, and 18,119 boundary-review cases.

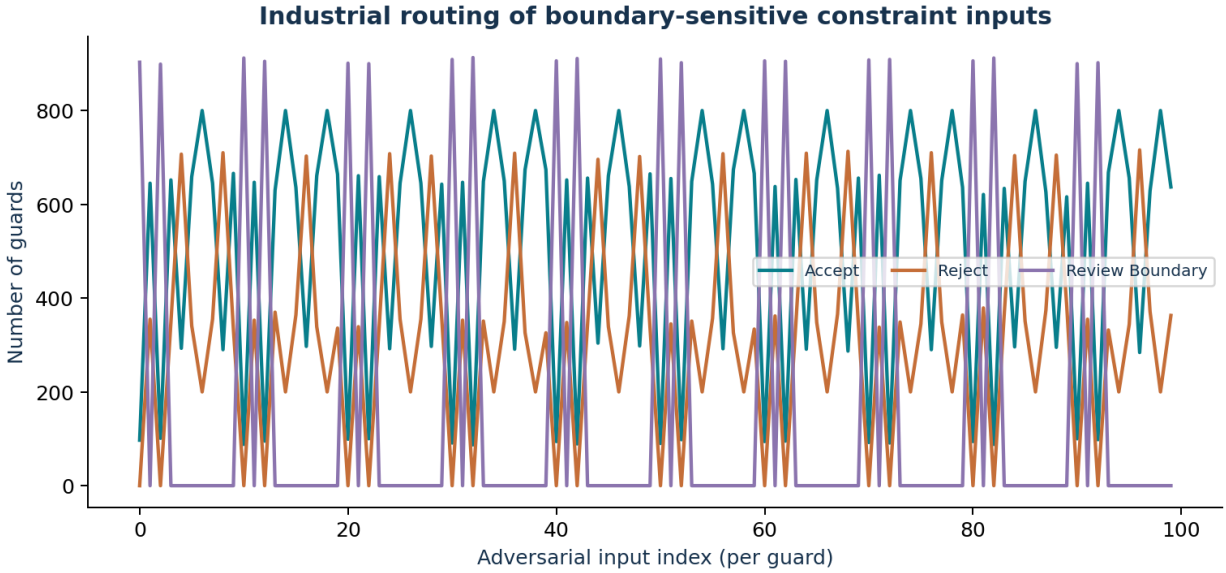


Figure 4. Routing outcomes across the fixed, boundary-focused input positions: explicit review behavior avoids treating near-threshold values as unconditional approvals.

7. Three-dimensional constraint geometry

A 61 x 61 numerical grid is evaluated directly with paceval for a representative elliptical guard. The signed margin provides an interpretable view of a candidate two-dimensional operating region: positive and negative heights represent admissible and non-admissible sample positions under the selected mathematical model.

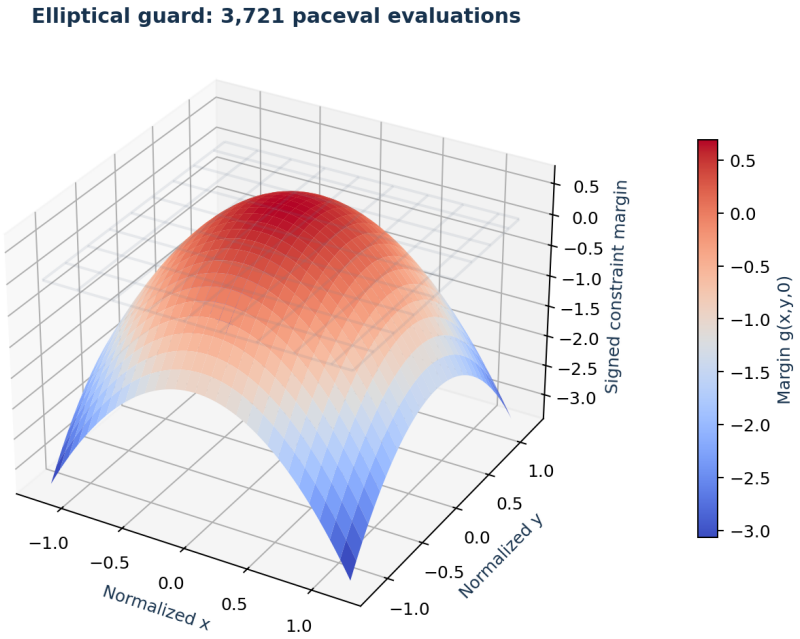


Figure 5. Three-dimensional mathematical margin surface from 3,721 genuinely evaluated paceval outputs.

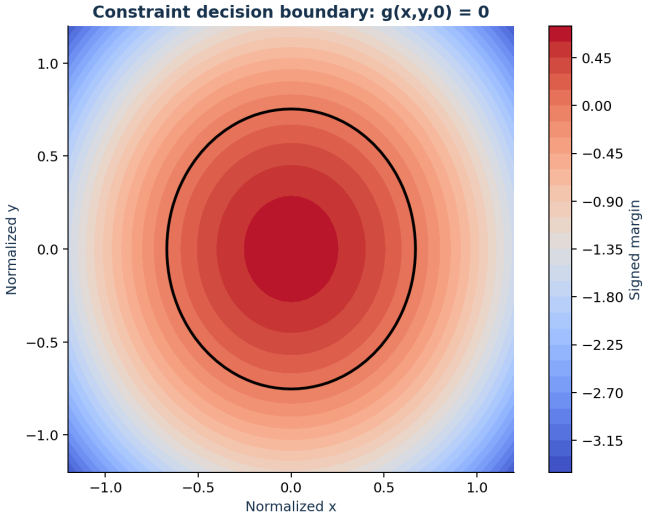


Figure 6. Contour visualization from the same recorded numerical values, with a zero-margin ellipse.

8. Constraint families and engineering interpretation

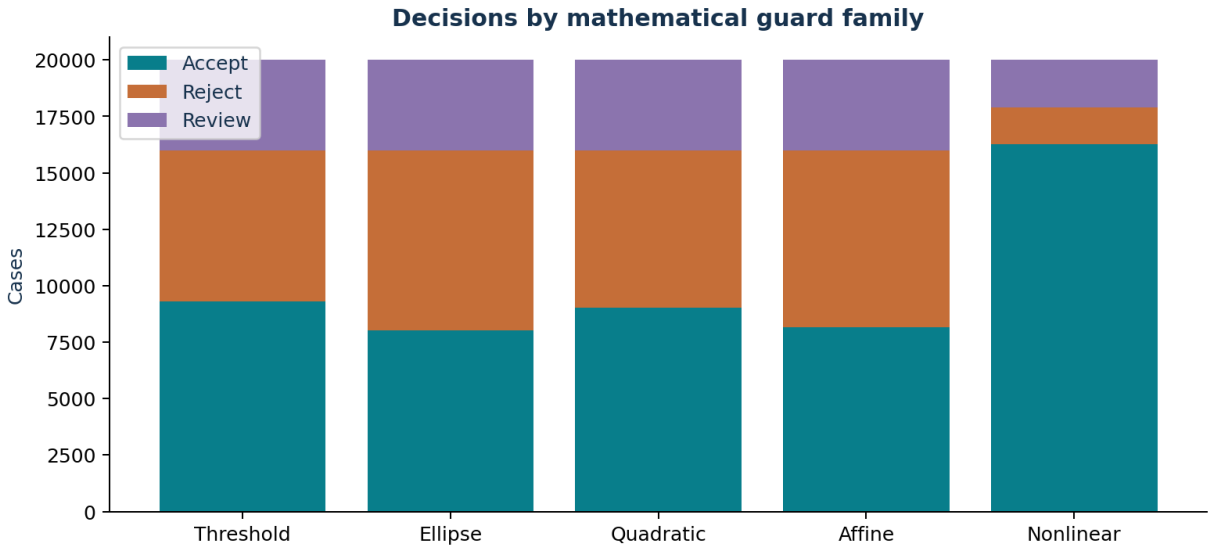


Figure 7. Decision class distribution grouped by the five mathematical model families.

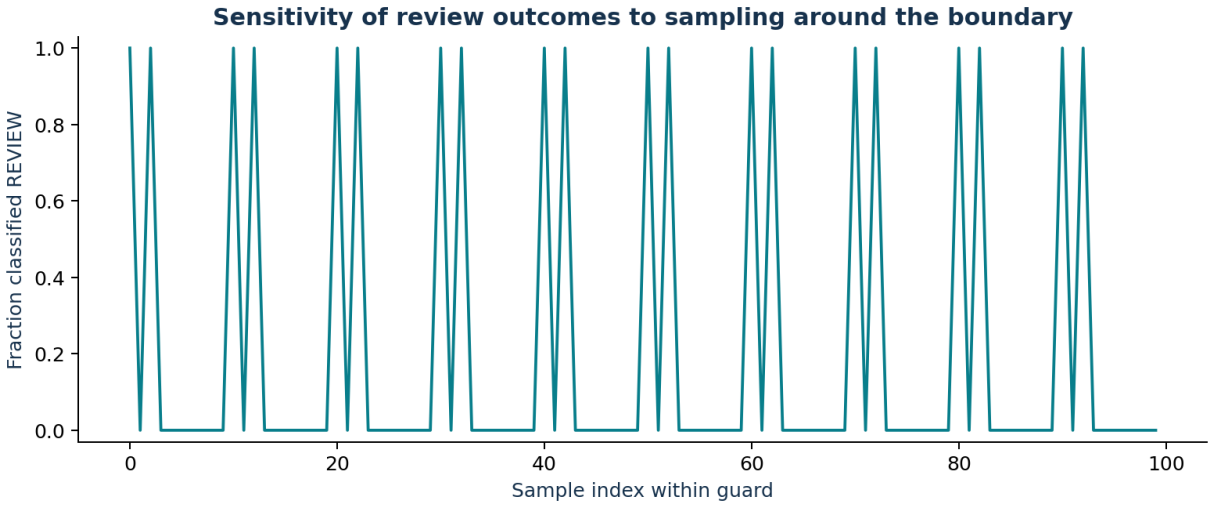


Figure 8. Fraction of boundary-review cases at each deterministic sample position.

Different shapes give different admissible regions: affine half-spaces, elliptical work envelopes, quadratic resource limits and nonlinear response boundaries. For a manufacturing application, the selected envelope would need to be derived from the actual plant and independently validated before influencing equipment permissions.

9. Reproducibility, limitations and sources

Rebuild the measured mathematical experiment

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release

-DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so

cmake --build build -j

ctest --test-dir build --output-on-failure

./build/agent_constraints --out reproduced
```

The repository contains the commented C++ application, the original paceval SDK C/C++ facade files, audit scripts, a frozen guard registry, 100,000 output decisions, a 3,721-point constraint surface and the scientific plots. The compatible shared paceval Linux runtime is supplied separately and is not redistributed.

Scientific limits

The guard expressions and samples are synthetic and dimensionless; they do not encode actual machine or robot safety limits. The program neither accepts free-form LLM code nor provides a theorem prover, SMT solver, certified interval arithmetic, a full network MCP implementation, or an industrial safety approval. The host review threshold is not a guaranteed error bound. Production use would additionally require real engineering specifications, typed schemas, authorization, physical interlocks, independent functional-safety validation and secure tool interfaces.

Selected technical sources (verified November 2026)

SMT-LIB 2.7 reference: https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-07-07.pdf

Z3 guide: arithmetic theories:
https://github.com/microsoft/z3guide/blob/main/website/docs-smtlib/02%20-%20theories/01%20-%20Arithmetic.md

NIST Generative AI Profile:
https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence

MCP tools specification, 2026-07-28:
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/tools.mdx

paceval API: https://paceval.com/api/

Conclusion

Persistent paceval mathematical graphs successfully evaluated a large set of specified, generated inequalities inside a controlled industrial-style decision workflow. The numerical results can support auditable routing decisions while authorization, hardware safety and formal universal verification remain separately defined responsibilities.